

第 2 章

数据库 DML 操纵语言

朱泽明

主要 DML 语句

www.qianrushi.com.cn

- ❖ INSERT
- ❖ SELECT
- ❖ UPDATE
- ❖ DEL

INSERT 插入

插入记录操作

www.qianrushi.com.cn

❖ 语法：

```
INSERT [INTO] <表名> [列名] VALUES <值列表>
```

❖ 叙述：

- 如果表名后面没写字段名，则默认是向所有的字段添加值，另外字符串值应该用‘ ’或“ ”引号括起来

❖ 举例 1：向 people 表中添加一条记录：

```
insert into people(name,age)
values("zhangsan",20);
```

插入案例

www.qianrushi.com.cn

❖ 创建一张学生信息表，往表中插入数据

```
create table students(  
    scode int not null auto_increment,  
    sname varchar(20) not null,  
    saddress varchar(20) default '未知',  
    sgrade int,  
    semail varchar(20),  
    ssex int,  
    primary key(scode)  
);
```

插入数据行 1

www.qianrushi.com.cn

注意事项 1: 每次插入一行数据，不可能只插入半行或者几列数据，因此，插入的数据是否有效将按照整行的完整性的要求来检验；

```
INSERT INTO Students (SName,SAddress,SGrade,SEmail,SSEX)  
VALUES ('小明') ❌
```

插入数据行 2

www.qianrushi.com.cn

注意事项 2：每个数据值的数据类型、精度和小数位数必须与相应的列匹配；

```
INSERT INTO Students (SName,SAddress,SGrade,SEmail,SSEX)  
VALUES ('张青裁','上海松江',ZQC,'ZQC@Sohu.com', '男')
```

插入数据行 3

www.qianrushi.com.cn

注意事项 4：如果在设计表的时候就指定了某列不允许为空，则必须插入数据；

```
INSERT INTO Students (SAddress,SGrade,SEmail,SSEX)  
VALUES ('上海松江',6,'ZQC@Sohu.com',0) ❌
```


插入数据行 4

www.qianrushi.com.cn

注意事项 6: 具有缺省值的列, 可以使用 DEFAULT (缺省) 关键字来代替插入的数值

```
INSERT INTO Students (SName,SAddress,SGrade,SEmail,SSEX)  
VALUES ('张青裁',DEFAULT,6,'ZQC@Sohu.com',0)
```

插入多行数据

www.qianrushi.com.cn

```
INSERT INTO <表名>( 列名 )  
VALUES (<列名值>), (<列名值>), (<列名值>)  
.....
```

```
INSERT STUDENTS (SName,SGrade,SSex)  
VALUES(' 测试女生 1',75,0),  
( ' 测试女生 2',77,0),  
( ' 测试女生 3',83,0),  
( ' 测试男生 1',81,1),  
( ' 测试女生 4',90,0),  
( ' 测试男生 2',94,1),  
( ' 测试女生 5',51,0),  
( ' 测试男生 3',53,1);
```

插入多行数据

www.qianrushi.com.cn

```
INSERT INTO <表名>( 列名 )  
SELECT <列名>  
FROM <源表名>
```

```
INSERT INTO TongXunLu ( ' 姓名 ' , ' 地址 ' , ' 电子邮件 ' )  
SELECT      SName,SAddress,SEmail  
FROM        Students
```

学生、课程、成绩插入数据

www.qianrushi.com.cn

❖ 学生信息

- '95001','洛燕妮','女','20','信息系'
- '95002','欧阳炎','男','19','计算机系'
- '95003','何东升','男','21','美术系'
- '95004','刘晨','男','18','计算机系'
- '95005','沙明华','女','21','美术系'
- '95006','欧阳宝宝','男','19','美术系'

❖ 课程信息

- 1,'JAVA'
- 2,'C'
- 3,'.NET'

❖ 成绩信息

- '95001',1,98 '95002',1,50 '95004',1,70
- '95003',4,95 '95005',4,40 '95006',4,77
- '95001',2,80 '95002',3,58 '95006',2,70

UPDATE 更新

更改记录操作

www.qianrushi.com.cn

❖ 语法：

```
UPDATE < 表名 > SET < 列名 = 更新值 >  
[WHERE < 更新条件 >]
```

❖ 叙述：

where 子句是判断语句，用来设定条件，限制只更新匹配的行，如果不带 where 子句，则更新所有行数据。

• 举例：

将 student 表中的所有学生名称为 "Alex" 的改为 "Tom":

```
update student set sname="Tom" where sname="Alex";
```

更新数据行

www.qianrushishi.com.cn

```
UPDATE Student SET SSEX = 0
```

```
UPDATE Student  
SET Sdept = '家政系'  
WHERE Sdept = '刺绣系'
```

```
UPDATE Sc  
SET Grade = Grade + 5  
WHERE Grade <= 95
```

DELETE 删除

❖ 语法：

DELETE FROM < 表名 > [**WHERE** < 删除条件 >]

❖ 叙述：此语句删除表中的行，如果不带 where 子句，则删除整个表中的记录，但是表不被删除。

❖ 举例：

删除 student 表中的所有年龄小于 18 岁的记录：

```
delete from student where sage<18;
```

补充说明

www.qianrushi.com.cn

❖ 删除学生编号为 '95005' 的学生信息包括成绩信息

```
delete from student,sc  
using student,sc  
where student.sno = sc.studentid  
and student.sno='95005';
```

❖ 说明：

表和字段的引用方式有两种：绝对引用和相对引用

- 绝对引用：数据库名 . 表名 (. 字段名)
- 相对引用：表名 . (字段名)

删除数据行

www.qianrushi.com.cn

学号	姓名	地址	
0010012	李山	山东定陶	
0010013	吴兰	湖南新田	
0010014	雷铜	江西南昌	
0010015	张丽鹃	河南新乡	
0010016	赵可以	河南新乡	

```
DELETE FROM 学员信息表
WHERE 学号 = '0010012'
```

科目	学号	分数	
数学	00100	88	
数学	00100	74	
语文	00100	67	
语文	00100	81	
数学	00100	98	

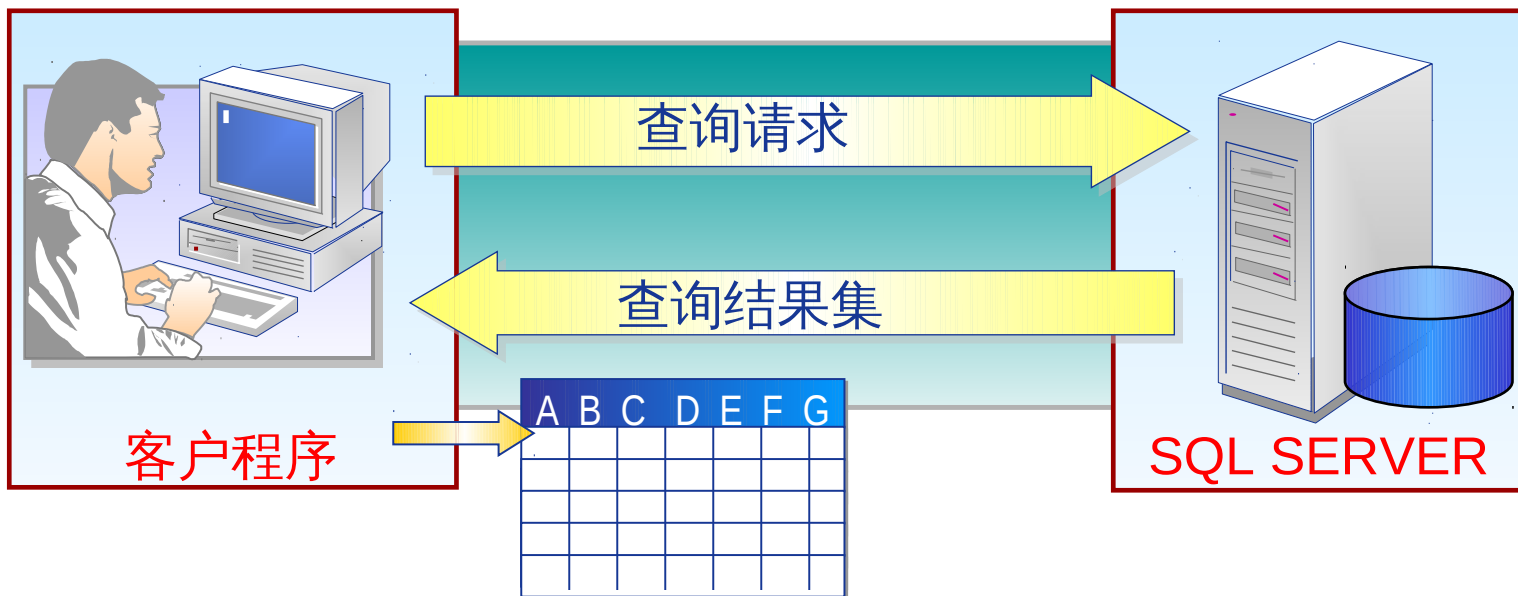
16

SELECT 查询

什么是查询？

www.qianrushu.com.cn

SELECT * FROM SALES



查询产生一个虚拟表，看到的是表形式显示的结果，但结果并不真正存储，每次执行查询只是现从数据表中提取数据，并按照表的形式显示出来

怎么查的？

www.qianrushi.com.cn

学员编号 学员姓名 地址 所在班级

```
SELECT * FROM Student  
WHERE 所在班级 = 'S202'
```

001 张明全 湖南长沙 S201

002 李菲 湖北宜昌 S202

003 于寄谦 甘肃天水 S204

004 刘国正 山东菏泽 S201

005 周接轮 台湾新竹 S204

006 巩小妹 香港龙湾 S203

007 巩大妹 香港龙湾 S203

008	002	李菲	湖北宜昌	S202	...
008	008	张明敏	北京顺义	S202	
009	010	罗林光	陕西临潼	S202	

SQL 语法

www.qianrushi.com.cn

```
SELECT [DISTINCT | DISTINCTROW | ALL]
select_expression,...
[FROM table_references
    [WHERE where_definition]
    [GROUP BY col_name,...] 集函数
    [HAVING where_definition]
    [ORDER BY {unsigned_integer | col_name |
formula} [ASC | DESC] ,...]
    [LIMIT [offset,] rows]
    [PROCEDURE procedure_name]
]
```

- ❖ **from** 子句：指定查询数据的表
- ❖ **where** 子句：查询数据的过滤条件
- ❖ **group by** 子句：对匹配 where 子句的查询结果进行分组
- ❖ **having** 子句：对分组后的结果进行条件限制
- ❖ **order by** 子句：对查询结果结果进行排序，后面跟 **desc** 降序或 **asc** 升序（默认）。
- ❖ **limit** 子句：对查询的显示结果限制数目
- ❖ **procedure** 子句：查询存储过程返回的结果集数据

查询举例

www.qianrushi.com.cn

SELECT < 列名 >
FROM < 表名 >
[WHERE < 查询条件表达式 >]
[ORDER BY < 排序的列名 >[ASC 或 DESC]]

列名称	表名	过滤条件	排序条件
SELECT SNo, SName, Sdept	FROM Student	WHERE SSEX = 0	ORDER BY SNo

SELECT SNo,SName,Sdept
FROM Students
WHERE SSEX = 0
ORDER BY SNo

- ❖ **举例 1**：查询全体学生的学号与姓名。
select sno,sname from student;
- ❖ **举例 2**：查询全体学生的详细记录。
select sno,sname,ssex,sage,sdept from student;
或 select * from student;
- ❖ **举例 3**：查全体学生的姓名及其出生年份。
select sname,2008-sage from student ;
- ❖ **举例 4**：查询全体学生的姓名、出生年份和所有系，要求用小写字母表示所有系名。
select sname,'year of birth: ',2008-sage,
lower(sdept) from student;
- ❖ **举例 5**：查询选修了课程的学生学号（去掉重复的记录）
select distinct studentid from sc;

- ❖ **举例 6**：查询全体学生的学号与姓名，用中文显示列名。
select sno as '编号',sname as '姓名' from student;
- ❖ **举例 7**：给表设置别名。
select s.sno,s.sname from student as s;
- ❖ **举例 8**：查询年龄在 20 以下的学生的姓名。
select sname from student where sage<20 ;
- ❖ **举例 9**：查询全体学生的姓名、年龄，要求按照年龄降序排序。
select sname,sage from student order by sage desc;
- ❖ **举例 10**：查询年龄最大的前 3 个学生的姓名和年龄，或第 4、5 个学生
select sname,sage from student order by sage desc
limit 3; 或 (limit 3,2)

使用集函数

www.qianrushi.com.cn

❖ SQL 提供的统计函数称为集函数。

❖ 主要的集函数：

- 记数函数：count(列名) 计算元素的个数
- 求和函数：sum(列名) 对某一列的值求和，但属性必须是整型
- 计算平均值：avg(列名) 对某一列的值计算平均值
- 求最大值：max(列名) 找出某一列的最大值
- 求最小值：min(列名) 找出某一列的最小值

❖ 举例 11： 查询学生总数。

```
select count(*) from student;
```

❖ 举例 12： 查询选修了课程的学生人数。

```
select count(distinct studentid) from sc;
```

❖ 举例 13： 查询 1 号课程的学生平均成绩。

```
select avg(grade) from sc where courseid=1;
```

❖ 举例 14： 查询 1 号课程的学生最高分和最低分。

```
select max(grade) as '最高分', min(grade) as '最低分'  
from sc where courseid=1;
```

❖ 举例 15： 查询每个学生的平均成绩。

```
select studentid, avg(grade) as '平均成绩' from sc  
group by studentid;
```

❖ 举例 16： 查询学生的平均成绩在 70 分以上的。

```
select studentid, avg(grade) as '平均成绩' from sc  
group by studentid having avg(grade)>70;
```

❖ 在 WHERE 子句中使用谓词：

BETWEEN AND ：在两数之间

NOT BETWEEN AND ：不在两数之间

IN < 值表 > ：是否在特定的集合里（枚举）

NOT IN < 值表 > ：与上面相反

LIKE ：是否匹配于一个模式

IS NULL （为空的）或 IS NOT NULL （不为空的）

REGEXP ：检查一个值是否匹配一个常规表达式（正则表达式）。

- ❖ **举例 17**：查询年龄在 20~23 岁（包括 20 岁和 23 岁）之间的学生的姓名、系别和年龄。

```
select sname,sdept,sage from student where sage  
between 20 and 23;
```

- ❖ **举例 18**：查询年龄不在 20~23 岁之间的学生姓名、系别和年龄。

```
select sname,sdept,sage from student where sage not  
between 20 and 23;
```

- ❖ **举例 19**：查询 '信息系'、'美术系' 和 '计算机系' 学生的姓名和性别。

```
select sname,ssex from student where sdept in ('信息  
系','美术系','计算机系');
```

- ❖ **举例 20**：查询学号为 95001 的学生的详细情况。

select * from student where sno like '95001' ;

等价于：select * from student where sno='95001';

- ❖ **举例 21**：查询所有姓刘学生的姓名、学号和性别。

select sname,sno,ssex from student where sname like '刘 %';

- ❖ **举例 22**：某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号。

select studentid,courseid from sc
where grade is null;

- ❖ **举例 23**：查所有有成绩的学生学号和课程号。

select studentid,courseid from sc
where grade is not null;

- ❖ **举例 24**：查询计算机系年龄在 20 岁以下的学生姓名。

select sname from student where sdept='计算机系' and
sage<20;

❖ **举例 25：** 查询信息系、美术系和计算机系学生的姓名和性别
select sname,ssex from student where sdept in ('信息系','美术','计算机系');

可改写为：

Select sname,ssex from student where sdept='信息系' or sdept='美术' or sdept='计算机系';

❖ **举例 26：** 查询选修了 3 号课程的学生学号及其成绩，查询结果按分数降序排列。

select studentid,grade from sc where courseid=3 order by grade desc;

❖ **举例 27：** 查询全体学生情况，查询结果按所在系的系号升序排列，同一系中的学生按年龄降序排列。

select * from student order by sdept,sage desc;

创建计算字段

www.qianrushi.com.cn

concat

```
mysql> select concat(stuName, ' is ', stuAge, ' years old')  
-> from student order by stuAge desc;
```

```
+-----+  
| concat(stuName, ' is ', stuAge, ' years old') |  
+-----+  
| 李四 is 32 years old |  
| 芒果瓜黄 is 20 years old |  
| 玫瑰红 is 18 years old |  
| 荔枝白 is 18 years old |  
| 香蕉强 is 17 years old |  
| 张紫柔 is 16 years old |  
| 张三 is 12 years old |  
| 西瓜率 is 12 years old |  
+-----+
```

创建计算字段

www.qianrushi.com.cn

concat

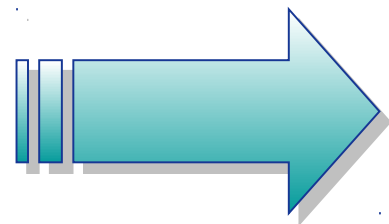
```
mysql> select concat(stuName, ' is ', stuAge, ' years old')  
-> as stuInfo  
-> from student order by stuAge desc;
```

```
+-----+  
| stuInfo                                |  
+-----+  
| 李四 is 32 years old                  |  
| 芒果瓜黄 is 20 years old              |  
| 玫瑰红 is 18 years old                |  
| 荔枝白 is 18 years old                |  
| 香蕉强 is 17 years old                |  
| 张紫柔 is 16 years old                |  
| 张三 is 12 years old                  |  
| 西瓜率 is 12 years old                |  
+-----+
```

多表查询—问题

www.qianrushi.com.cn

学员内部测试成绩查询的每次显示的都是学员的编号信息，
因为该表中只存储了学员的编号；
实际上最好显示学员的姓名，而姓名存储在学员信息表；
如何同时从这两个表中取得数据？



多表查询—分类

www.qianrushi.com.cn

❖ 连接查询

- 同时涉及多个表的查询称为连接查询
- 用来连接两个表的条件称为连接条件

❖ 内连接 (INNER JOIN)

❖ 外连接

—— 左外联结 (LEFT JOIN)

—— 右外联结 (RIGHT JOIN)

外连接与普通连接的区别

- 普通连接操作只输出满足连接条件的元组
- 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出：

不支持全连接（笛卡尔乘积）

多表查询—内连接 -1

www.qianrushi.com.cn

Students

SName	SNo
梅超风	1
陈玄风	2
陆乘风	3
曲灵风	4

多少行？

C.CourseID,C.Grade

AS S

S C

> S.SNo

3	002	76
3	003	81

查询结果

猜一猜：这样写，

```
SELECT S.SName  
From Student  
INNER JOIN S  
ON C.StudentID
```

SName	CourseID	Grade
梅超风	001	97
陈玄风	001	89
陈玄风	002	67
陆乘风	002	76
陆乘风	003	81

?

多表查询—内连接 -2

www.qianrushi.com.cn

```
SELECT Students.SName, Sc.CourseID, Sc.Grade  
FROM Students,Sc  
WHERE Students.SNo = Sc.StudentID
```

多表查询—三表连接

www.qianrushi.com.cn

SELECT

S.SName AS 姓名, CS.CName AS 课程, C.Grade AS 成绩

FROM Students AS S

INNER JOIN Sc AS C ON (S.SCode = C.StudentID)

INNER JOIN Course AS CS ON (CS.CourseID = C.CourseID)

多表查询—左外连接

www.qianrushi.com.cn

```
SELECT      S.SName,C.CourseID,C.Grade
From        Students AS S
LEFT JOIN   Sc AS C
ON          C.StudentID = S.SNo
```

陈玄风	2	2	001	89
陆乘风	3	2	002	67
曲灵风	4	3	002	76

猜一猜：这样写，返回的查询结果是一样的吗？

```
SELECT      S.SName,C.CourseID,C.Grade
From        Sc AS C
LEFT JOIN   Students AS S
ON          C.StudentID = S.SNo
```

陈玄风	002	67
陆乘风	002	76
陆乘风	003	81
曲灵风	NULL	NULL

多表查询—右外连接

www.qianrushi.com.cn

```
SELECT Titles.Title_id, Titles.Title, Publishers.Pub_name  
FROM titles  
RIGHT OUTER JOIN Publishers  
ON Titles.Pub_id = Publishers.Pub_id
```

❖ 子查询：

- 将一个查询块嵌套在另一个查询块的 WHERE 子句或 HAVING 短语的条件中的查询称为子查询。一个 SELECT-FROM-WHERE 语句称为一个查询块。
- 例子：

```
create table emp (empno int,ename  
char(20),salary int,deptno int,mgr int)default  
charset = 'utf8';
```

```
insert into emp values(1,'Zyz',100,1,NULL),  
(2,'Lacus',50,1,1),(3,'Www',1000,2,4),  
(4,'Lacus',2000,2,NULL),(5,'Xxx',3000,3,4);
```

一．独立子查询：

1. 独立多值子查询（子查询返回的是数据集）

例如：

哪些人是其他人的经理？

```
select empno,ename from emp where empno in  
(select mgr from emp where mgr is not null);
```

当子查询返回多行时

all 所有的 any 任意一个

谁比所有的 Lacus 工资高

```
select empno,ename,salary from emp where  
salary >all(select salary from emp where ename  
= 'Lacus');
```

谁比任意一个 Lacus 工资高

```
select empno,ename,salary from emp where  
salary >  
any(select salary from emp where ename =  
'Lacus');
```

- 例如：
- 查询选修了课程名为“Java”的学生学号和姓名

```
select sno,sname from student
where sno in
(select studentid from sc
where cno =
(select cno from course
where cname='Java')
);
```

使用子查询

www.qianrushi.com.cn

2. 独立单值子查询（独立标量子查询）子查询返回的是单个值，而不是数据集

查询出工资比 Zyz 高的员工的编号、姓名、工资

```
select empno,ename,salary from emp where salary >  
(select salary from emp where ename = 'Zyz');
```

注：使用单值独立子查询时，一定要保证子查询返回的是单个值，而不是数据集；若返回的是数据集，则会报错。

使用子查询

www.qianrushi.com.cn

附：子查询和联接查询的选择

返回结果位于不同表中的列时，使用联接查询；

同一张表中的列时，使用子查询。

组合查询

www.qianrushi.com.cn

MySQL 允许执行多个查询语句，并将结果作为单个查询结果集返回。

需要使用到组合查询的情况：

- 1.在单个查询中从不同的表返回类似结构的数据
- 2.对单个表执行多个查询，按单个查询返回数据

组合查询

www.qianrushi.com.cn

找出年龄为 18 岁或者是计算机系或者是音乐系的学生

```
mysql> select stuName from student
-> where stuAge = 18 union
-> select stuName from student
-> where stuDepart in
-> ('计算机系', '音乐系');
```

```
+-----+
| stuName |
+-----+
| 张紫柔  |
| 玫瑰红  |
| 荔枝白  |
| 香蕉强  |
| 芒果瓜黄|
+-----+
```

组合查询

www.qianrushi.com.cn

找出年龄为 18 岁或者是计算机系或者是音乐系的
学生

```
mysql> select stuName from student
-> where stuAge = 18 union all
-> select stuName from student
-> where stuDepart in
-> ('计算机系', '音乐系');
```

stuName
张紫柔
玫瑰红
荔枝白
张紫柔
香蕉强
荔枝白
芒果瓜黄

union 会自动去重，而 union all 会将所有的都显示出来

组合查询

www.qianrushi.com.cn

组合查询之后的排序

```
mysql> select stuId, stuName from student  
-> where stuAge > 18 union  
-> select stuId, stuName from student  
-> where stuDepart in ('计算机系', '音乐系')  
-> order by stuId;
```

+	-	-	-	-	+	-	-	-	-	-	-	+
	stuId		stuName									
+	-	-	-	-	+	-	-	-	-	-	-	+
	10002		李 四									
	10003		张 紫 柔									
	10004		香 蕉 强									
	10007		荔 枝 白									
	10008		芒 果 瓜 黄									
+	-	-	-	-	+	-	-	-	-	-	-	+

组合查询

www.qianrushi.com.cn

union 使用规则：

1. union 必须由两条或者两条以上的 select 语句组成，语句之间用 union 关键字分隔（若组合 4 个 select 语句，则需要使用 3 个 union 关键字）
2. union 每个查询中必须包含相同的列、表达式或者聚集函数
3. 列数据类型必须兼容：类型不必完全相同，但必须是 DBMS 可以隐式转换的类型

组合查询

www.qianrushi.com.cn

小结

利用 union 可以将多条查询语句的结果作为一条组合查询返回，极大的简化了复杂的 where 条件子句，简化从多个表中检索数据的工作。

我们来创建一个副本：

```
mysql> CREATE TABLE grades SELECT * FROM marks;  
Query OK, 5 rows affected (0.04 sec)  
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM grades;
```

id	fname	lname	math	sci	lit
1	john	Doe	36	73	34
2	Sarah	Short	23	67	55
3	Joe	Cool	75	82	89
4	Mark	Wumba	58	72	15
5	Tom	Thumb	0	12	0

5 rows in set (0.01 sec)

❖ 可以通过在 SELECT 查询中指定字段来限制出现在附表中的字段

```
mysql> CREATE TABLE students SELECT fname,lname FROM marks;  
Query OK, 5 rows affected (0.01 sec)  
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM students;
```

```
+-----+-----+  
| fname | lname |  
+-----+-----+  
| john  | Doe    |  
| Sarah | Short  |  
| Joe   | Cool   |  
| Mark  | Wumba  |  
| Tom   | Thumb  |  
+-----+-----+  
5 rows in set (0.00 sec)
```



❖ 通过使用 SELECT 语句创建已存在表的空副本，并且返回一个空结果集。

```
mysql> CREATE TABLE g2 SELECT * FROM marks WHERE 0 = 1;  
Query OK, 0 rows affected (0.01 sec)  
Records: 0  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM g2;  
Empty set (0.00 sec)
```

```
mysql> DESC g2;
```

Field	Type	Null	Key	Default	Extra
id	int(11)			0	
fname	char(50)	YES		NULL	
lname	char(50)	YES		NULL	
math	int(11)	YES		NULL	
sci	int(11)	YES		NULL	
lit	int(11)	YES		NULL	

6 rows in set (0.02 sec)